HelixQA

HelixQA

Anti-bluff QA orchestration — autonomous, cross-platform sessions where every PASS carries captured evidence that a real user can use the feature.

Summary

HelixQA is an anti-bluff QA orchestration framework for cross-platform testing (Android, Android TV, Web, Desktop) that combines YAML test banks, real-time crash detection, step-by-step evidence capture, and LLM-plus-computer-vision autonomous QA sessions to prove features genuinely work end-to-end. It is the Constitution's mandated QA test-type (§11.4.169).

Short description

An anti-bluff QA orchestrator (Go) that runs written test banks and fully-autonomous, LLM-and-vision-driven QA sessions across platforms — detecting crashes, validating each step against captured evidence (screenshots, logcat, video, stack traces), and auto-generating evidence-rich tickets for AI fix pipelines.

Long description

HelixQA is a Go framework whose single, uncompromising design centre is the Constitution's §11.4 Operative Rule: the bar for shipping is not "tests pass" but "users can use the feature," so every PASS it emits must carry positive runtime evidence captured during execution — no evidence, no green, no exceptions. It runs two complementary modes that together cover both the scripted and the unknown. First, **written test banks** — YAML suites of `TC-XXX` cases with platform targeting, priority, ordered steps (name/action/expected), tags, and documentation refs — executed with per-step validation, real-time crash/ANR detection (ADB for Android, process monitoring for web/desktop),

centralized evidence collection, and auto-generated Markdown tickets already shaped for downstream AI fix pipelines. Second, a **fully-autonomous QA session** that hands the app to LLM-powered agents and computer vision and lets them drive it unattended across four disciplined phases: setup (select LLMs, build a feature map from project docs, spawn CLI agents, initialise the vision engine), doc-driven verification that walks every documented feature, curiosity-driven exploration that deliberately pokes at edge cases and undocumented behaviour, then report-and-cleanup into Markdown/HTML/JSON with every finding linked to video-timestamped evidence.

Crucially, it does not grade its own homework: it integrates four external Go submodules (LLMsVerifier, LLMOrchestrator, VisionEngine, DocProcessor) and reuses the shared `challenges` and `containers` infrastructure, so the component that navigates the app is not the component that judges whether it worked. Its own suite is held to exactly the same bar it enforces on others via `make anti-bluff` (static scan + behaviour-anchor manifest + mutation ratchet) and an 8-phase orchestrator Challenge with a built-in §1.1 mutation. A 15-row test-type coverage matrix nails every advertised capability to a concrete executable asset and a specific captured-evidence shape — so the framework's claims about itself are as evidence-bound as the verdicts it hands to the products it tests.

Why we built it

Conventional QA green-lights on "the assertion passed," which is exactly how the failure class the Constitution calls *bluffing* slips through — a feature reported working while it is broken for the real user. HelixQA was built to make that impossible for QA specifically: it refuses to score a PASS without physical evidence (screenshot, logcat, video, stack trace, report) captured under real execution, and it treats a green summary line with no such evidence as a critical defect equal to a missing feature. It also solves the labour problem — comprehensive manual QA across many platforms does not scale — by making the sessions fully autonomous.

Why it's a game-changer

It fuses two things that almost never live in the same tool: rigorous, evidence-backed QA gating and autonomous, self-driving exploration. An LLM-plus-vision agent opens the *actual* app, verifies every documented feature, goes hunting for the undocumented bugs no one wrote a test for, *and* produces a court-quality evidence trail while it does it — so "we tested it" is replaced by "here is the video, here is the logcat, here is the

ticket.” And because it is the Constitution-named QA submodule, adopting it doesn’t upgrade QA honesty for one team — it raises the floor for every consuming product in the family in a single move.

What’s innovative

- **Anti-bluff evidence contract** — every check’s PASS is bound to captured runtime evidence; a green CI line is treated as necessary but never sufficient, and a green summary with no evidence is scored as a critical defect.
- **Autonomous doc-driven + curiosity-driven exploration** — it verifies every documented feature *and* then goes off-script, probing the edge cases real users hit (empty inputs, rapid interactions, undocumented paths) that no hand-written suite anticipated.
- **Vision oracle** — GoCV mechanical vision plus the LLM Vision API literally *sees* the running UI on-screen, catching visually-broken states that token- and property-level assertions sail straight past.
- **Structure-not-prose test banks** — bank strings describe structure and drive LLM-generated question prompts at runtime (CONST-046), so a single bank works across locales instead of shattering the moment UI text is translated.
- **Tickets built for AI fix pipelines** — auto-generated Markdown issues arrive with the full evidence bundle attached, ready to hand straight to a downstream repair agent rather than a human triager.

How it is used across all products (the powers it gives)

As a **mandatory quality pillar** (Constitution §11.4.169 names the `helix_qa` submodule as one of the required test types), HelixQA gives every product in the family the same set of powers:

- **Autonomous QA sessions:** a single `helixqa autonomous --project … --platforms android,desktop,web` turns loose an LLM-plus-vision agent that drives real apps unattended toward a coverage target, emitting reports, tickets, and videos without a human in the loop.
- **Test banks / suites:** YAML banks (round-219 floor of ≥30), platform-targeted, priority-ranked, and traceable line-by-line back to the docs they verify.
- **Captured evidence:** screenshots, logcat, video, stack traces, and a full timeline — centralized and linked from every report, so any verdict can be replayed and audited after the fact.

- **Independent verdicts (§11.4.141 independence principle):** its LLM-powered `issuedetector` and vision oracle judge the running app's behaviour independently of the agent that navigated it, structurally ruling out the classic failure where a system marks its own work correct.
- **Gate + mutation ratchet:** `make qa-all` / `make anti-bluff` and `challenges/scripts/helixqa_orchestrator_challenge.sh` (8 phases, built-in §1.1 mutation) keep HelixQA's own honesty continuously proven — and there is deliberately no `--skip-helixqa` escape hatch to switch the discipline off under deadline pressure.

Biggest technical challenges & how we solved them

- **Preventing false positives in QA itself** — the tool that catches bluffs must not become one → every step is validated against captured evidence, a PASS without evidence is scored as a defect rather than a pass, and a behaviour-anchor manifest ties each advertised capability to an executable test (CONST-035) so no capability can be claimed without something that exercises it.
- **Driving heterogeneous platforms from one brain** — Android, Android TV, Web, and Desktop share no input model → a single `navigator` package abstracts over platform-specific ActionExecutors (ADB, Playwright, X11) and per-platform crash detectors (android/web/desktop), so the orchestration logic is written once and the platform differences stay at the edges.
- **Making autonomous agents useful, not chaotic** — an unsupervised LLM loose in an app can wander forever → LLMsVerifier scores and selects the right models, LLMOrchestrator manages the headless CLI agents (opencode, claude-code, gemini, junie, qwen-code), DocProcessor builds the feature map that gives exploration a target, and VisionEngine keeps every decision grounded in the real pixels on screen rather than the model's imagination.
- **Localization-safe banks** — a suite that hardcodes English UI text breaks in fifteen languages → banks describe structure only, and the user-facing prompt text is LLM/resource-loaded at runtime (CONST-046), so the same bank verifies the same behaviour regardless of locale.
- **Proving the gates aren't shams** — an anti-bluff gate that can't itself fail is the ultimate bluff → paired §1.1 mutations strip a type's evidence-capture or anti-bluff assertion and require the gate to FAIL, and a mutation ratchet stops that guarantee from silently eroding over time.

Tech stack

- **Go 1.24+ orchestrator** — *why:* QA has to run anywhere the products do, so a single statically-linked, fast, portable binary beats a runtime-heavy alternative; *how:* one `cmd/helixqa` CLI exposing composable subcommands `run` / `list` / `report` / `autonomous` / `version`.
- **YAML test banks (`pkg/testbank`)** — *why:* suites should be declarative and readable, editable by humans without touching Go; *how:* `version`/`name`/`test_cases[]` with `id`, `category`, `priority`, `platforms`, ordered `steps[]`, and `documentation_refs[]` for traceability back to the feature docs.
- **Crash/ANR detectors (`pkg/detector`)** — *why:* the failures that matter most are the ones that happen live, mid-interaction, not in a post-hoc assertion; *how:* ADB (`pidof`/`logcat`/`screencap`) for Android and `pgrep` for web/desktop, watching the process while the test drives it.
- **Evidence collection (`pkg/evidence`, `pkg/session`)** — *why:* the anti-bluff contract is only real if every PASS is backed by physical proof; *how:* screenshots, logcat, video, and stack traces captured into a `SessionRecorder` timeline that every report links back to.
- **Autonomous session (`pkg/autonomous`, `pkg/navigator`, `pkg/issuedetector`)** — *why:* comprehensive manual QA across four platforms does not scale, so the exploration itself has to self-drive; *how:* a 4-phase `SessionCoordinator` plus ActionExecutors (ADB/Playwright/X11) and LLM bug detection spanning visual, UX, accessibility, and functional defects.
- **External submodules** — *why:* reuse and decoupling (CONST-051), and — critically — separation of the navigator from the judge; *how:* LLMsVerifier (model scoring), LLMOrchestrator (headless CLI agents), VisionEngine (GoCV + LLM Vision), DocProcessor (feature-map/coverage), each an independently-owned component.
- **Anti-bluff gates + mutation ratchet** — *why:* to hold HelixQA to the exact §1.1 covenant it enforces on everything else; *how:* a `make anti-bluff` scan plus a behaviour-anchor manifest and mutation ratchet, with `helixqa_orchestrator_challenge.sh` as an 8-phase end-to-end validator.
- **15-row coverage matrix (`docs/test-coverage.md`)** — *why:* CONST-050(B) mandates a closed, fully-accounted-for test-type set with no gaps; *how:* each row is bound to a concrete executable asset and a specific captured-evidence shape, so coverage is a checked fact rather than a claim.

Status & honesty notes

- **Status: beta.** Actively developed (README status banner round 219). Held to its own anti-bluff bar.

- **License: Apache-2.0.** Install: `go install digital.vasic.helixqa/cmd/helixqa@latest`.

Priority tier: Helix-primary — a mandatory quality/anti-bluff pillar of how the Helix family verifies that features genuinely work.